CSC 4553

Fall 2026 - Day 18

Admin

- Exam 2 next week
- Assignment 3 posted

Paging

Questions to consider

- How does paging eliminate external fragmentation while supporting multiple processes?
- How does the hardware (MMU) translate virtual addresses to physical addresses?
- What information must the OS maintain to support paging?

Paging

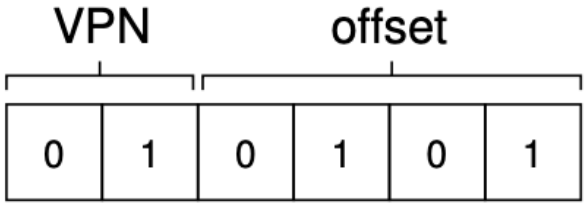
- Contiguous allocation has many issues; what should we do?
  - → Be non-contiguous!
- **Paging**: split memory into fixed-size blocks
  - → Eliminates external fragmentation
  - → Needs hardware support (MMU) to translate addresses via a page table

Paging (cont'd)

- Jargon!
  - → Physical memory: **frames**
  - → Logical memory: **pages**
  - → Backing store: **blocks**
- Paging maps pages to frames
- Pages are logically contiguous; frames and blocks need not be
- Typically |frames| = |pages| = |blocks|
  - → Most common: 4KB

Paging (cont'd)

- Addresses have two parts: **page number** and **page offset**
  - → Page number: index to a page
  - → Offset: byte within that page
- Processes use virtual addresses. Which part do we translate?
  - → Page number only; offset stays the same

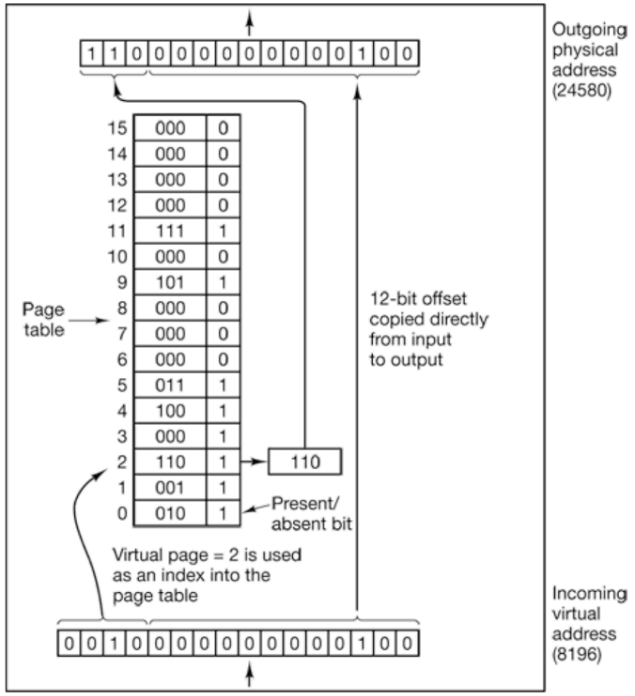


Paging (cont'd)

- The **page table** maps virtual page numbers to physical frames
  - → The MMU uses it on every memory access
- Transparent to the process; the OS controls the page table
- One page table *per process*
  - → Where should it be stored?
  - → In memory; pointer held in the PCB

Page tables

- Address space 2^m bytes, page size 2^n bytes
  - → High $m - n$ bits: page number
  - → Low n bits: offset
- This example: 2^4 = 16 pages, 2^{12} = 4KB pages



Examples

- 32-bit virtual address, 8-bit offset. How many pages? Page size? Total size?
  - → 2^{24} (~16M) pages, 256B pages, 4GB total
- 256 pages, 8KB each
  - → 8-bit index, 13-bit offset, 21 bits = 2MB total

Translate an address

- 16-bit virtual addresses, 4KB pages
- Page table:

Page	Frame
0	2
1	5
2	not present
3	1

- Translate: **0x1A3F**, **0x3FFF**, **0x2004**

Page tables (cont'd)

- Page size depends on system usage
  - → Smaller: finer granularity, but bigger page table
  - → Larger: fewer, bigger disk transfers and fewer TLB misses
- How do pages affect fragmentation?
  - → **No external fragmentation**
  - → But now internal fragmentation
  - → On average: $1/2$ page per process

Page table entries

- What goes into a page table entry?
  - → Page frame number (duh)
  - → **Present/absent**: is the frame number valid?
  - → **Protection**: what access is allowed
  - → **Modified/Dirty**: written since loaded?
  - → **Referenced**: used recently?
  - → **Caching disabled**: bypass the cache?
- What doesn't go in?
  - → Where the page lives on disk
    - Only what the hardware needs for translation

Size the page table

- Assume 4 KB pages and 4-byte page table entries. How big is a single, flat page table for…
  1. A 32-bit address space?
  2. A 64-bit address space?
- 100 processes on the 32-bit machine — total memory just for page tables?

  → What does this tell you about storing page tables in RAM?

Paging summary

- Pages allow us to:
  - → Load processes at finer granularity
  - → Avoid external fragmentation
  - → Swap individual pages (“paging”)
- New questions:
  - → Which pages are in memory?
  - → When we need a frame, which page goes?
  - → Which pages are modified and need writing to disk?

What isn’t clear?

Comments? Thoughts?

Page replacement

Questions to consider

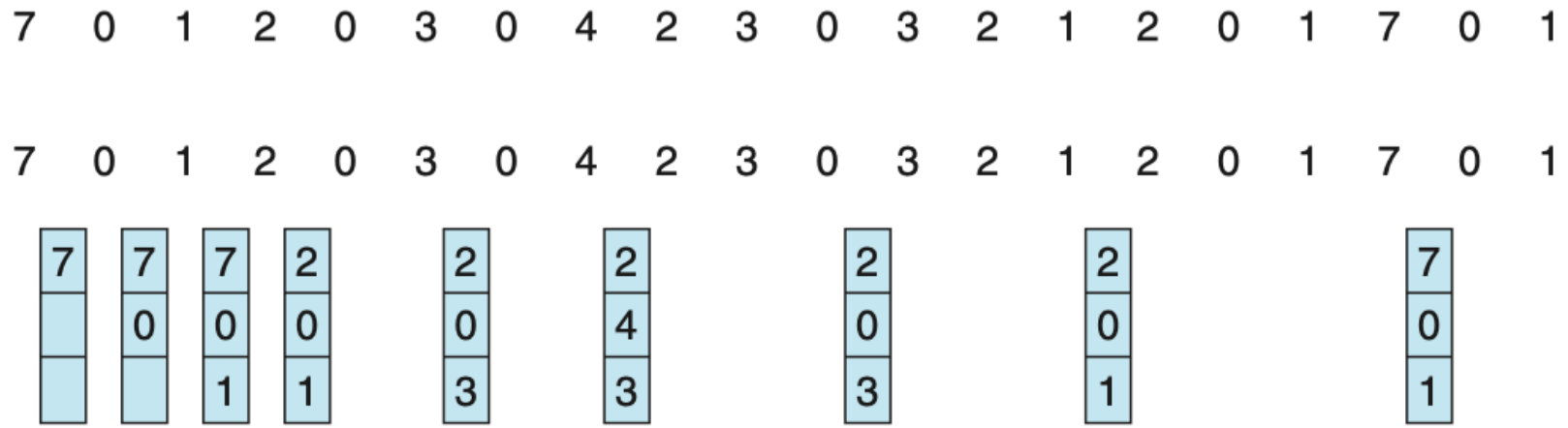
- What makes a good page replacement strategy, and why?
- How can we approximate optimal page replacement without knowing the future?
- What are the practical implementation challenges for different page replacement algorithms?

Page replacement

- Memory is full and a process needs a page: evict a frame (**page replacement**)
- The evicted page is the **victim**
- Costly: 1 I/O for a clean victim, 2 for a dirty one
- **Clean pages are preferred victims**
  - → Dirty pages need a write-back first
- Downside of evicting dirty pages?
  - → Forces the write at the worst time
  - → Dirty pages were recently active

Page replacement algorithms: OPT

- Optimal (OPT): evict the page that *won't be used for the longest time*
- Impossible (like SJF), but a useful benchmark
- Example: 3 frames, this reference pattern:



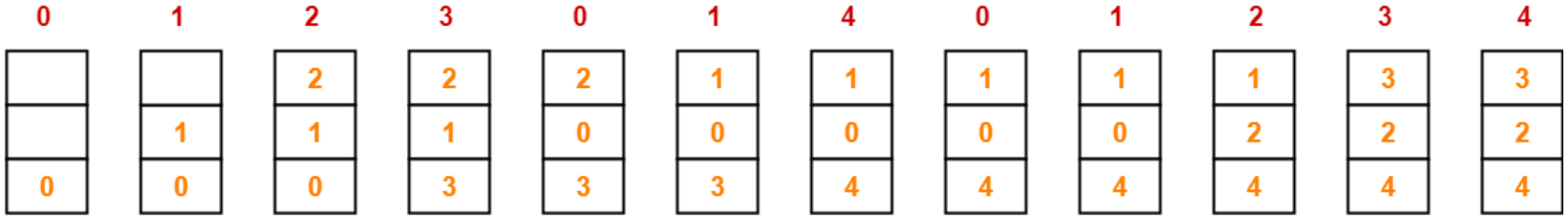
Page replacement algorithms: FIFO

- First-In-First-Out (FIFO): evict the oldest page
- Downsides?
  - → Ignores use: may evict important pages
- **Belady's anomaly**: more frames can mean more page faults
- Try this pattern with 3 frames, then 4:

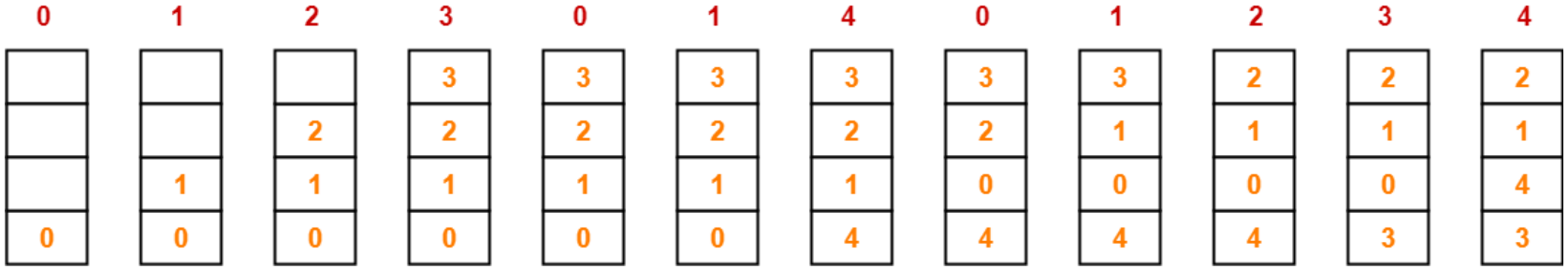
0 1 2 3 0 1 4 0 1 2 3 4

Belady's anomaly

- FIFO lacks the stack property, so more frames can mean more faults



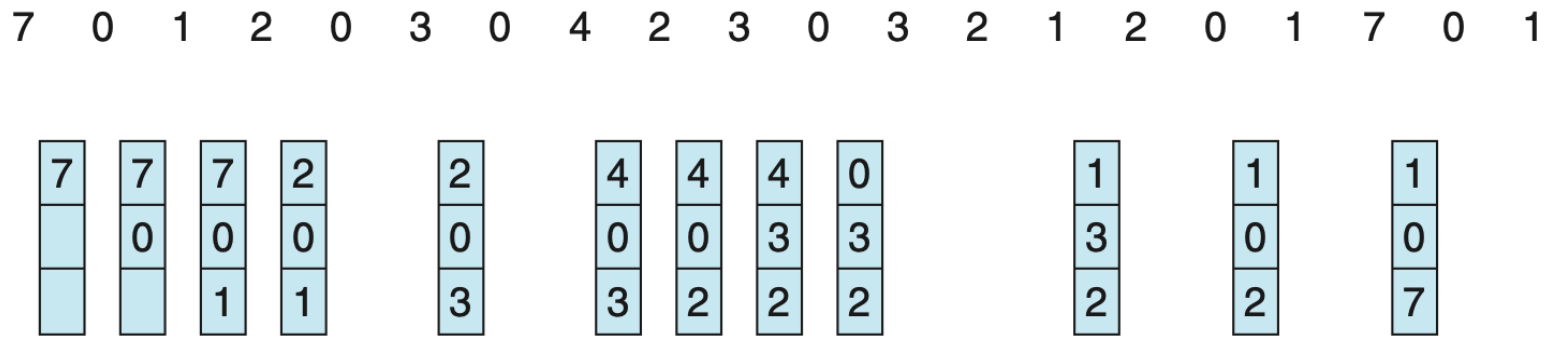
- Number of page faults with 3 frames = 9



- Number of page faults with 4 frames = 10

Page replacement algorithms: LRU

- We can't see the future. What can we do?
  - → Look at the past
- Least Recently Used (LRU): evict the page unused for the longest time
  - → OPT looking backwards
  - → The evicted page might be needed right away



LRU (cont'd)

- LRU does not exhibit Belady's anomaly
- How can we implement LRU?
  - → Timestamp per entry: space, plus a search
  - → Ordered list: updates on every access
  - → Both impractical without special hardware
- Approximate it with a **reference bit**
  - → Set when the page is referenced
  - → Cleared periodically

Page replacement algorithms: others

- Second Chance: FIFO queue + reference bit
  - → Bit 0: replace the page
  - → Bit 1: clear it, move page to the back
  - → Downside?
    - Constantly moving pages in the list
- Clock: same idea, circular list
  - → Hand points at the oldest page
  - → Bit 0: replace; bit 1: clear and advance
  - → A good approximation of LRU

Page replacement algorithms: others (cont'd)

- Least Frequently Used (LFU): evict lowest use count
  - → Downside?
    - Heavy early use keeps a page forever
- Most Frequently Used (MFU): evict highest count
  - → Downside?
    - Evicts the pages you use most
- Both are poor: expensive and don't approximate OPT

Page replacement algorithms: summary

- OPT is optimal but impossible; LRU approximates it well
- Many algorithms, with different performance and hardware needs
- e.g., Linux: active/inactive lists (2Q-like)
  - → Driven by the hardware referenced bit
- Page replacement shows up at every level of the memory hierarchy
  - → We focus on disk/RAM: the biggest gap in access time

What isn’t clear?

Comments? Thoughts?